Banco de Dados

Índices

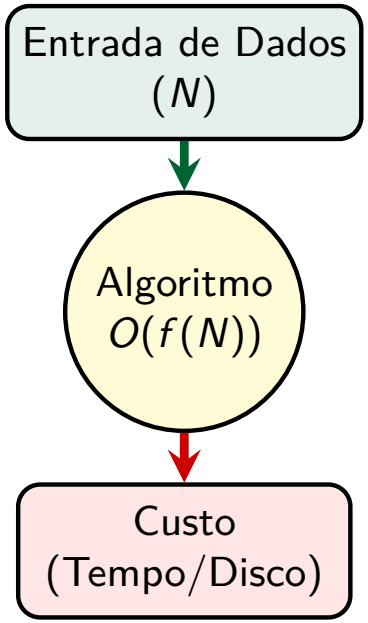
Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



Revisão 1: O que é Complexidade Computacional?

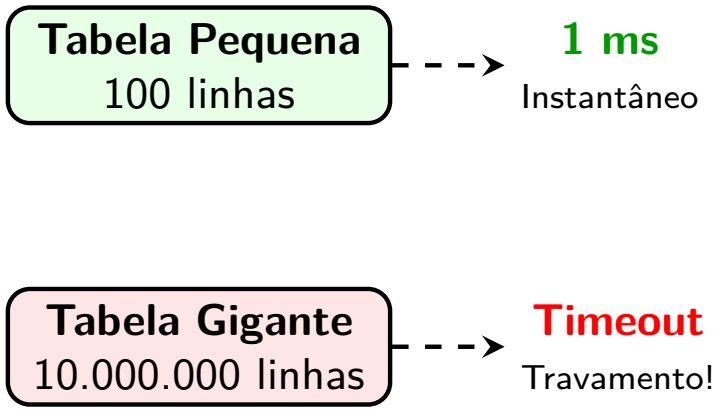
- Uma forma matemática de analisar como um algoritmo se comporta conforme a quantidade de dados (N) aumenta.
- Usamos a **Notação Big-O** ($O()$) para descrever o limite superior.

- Não mede o tempo exato em segundos, mas o **crescimento das operações**.
- Em BD, traduz-se no número de **acessos ao disco**.



Revisão 2: Por que isso importa em BD?

- Bancos de dados relacionais lidam com **milhões** ou **bilhões** de registros.
- Um algoritmo que parece rápido com 100 linhas pode travar o sistema com 1.000.000 de linhas.
- Criar um **Índice** é trocar um algoritmo ineficiente por um super otimizado!

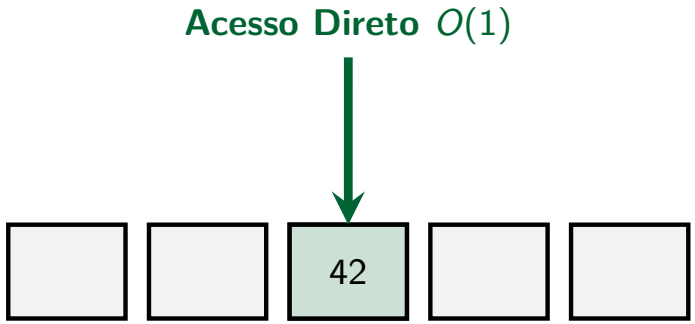


Revisão 3: $O(1)$ - Complexidade Constante

- O tempo de execução é **sempre o mesmo**, não importa o volume de dados.
- **Cenário:** O "Santo Graal". Acesso direto via endereço físico ou Hash.

Consulta $O(1)$:

```
SELECT * FROM clientes
WHERE rowid = 42;
```

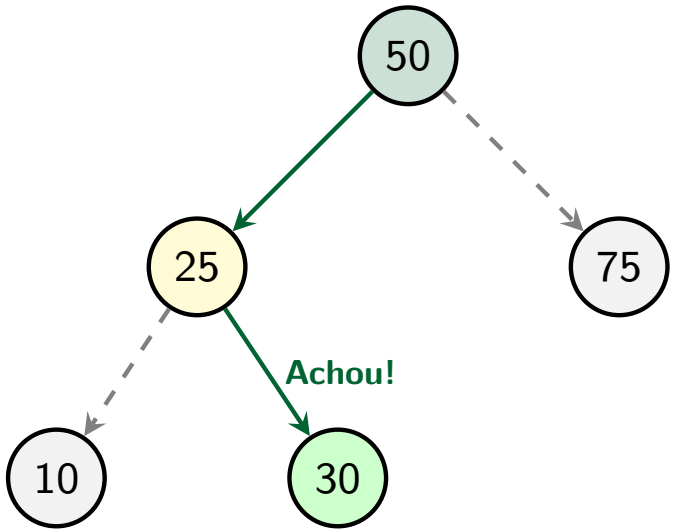


Revisão 4: $O(\log N)$ - Complexidade Logarítmica

- O tamanho do problema reduz pela metade a cada passo.
- **Cenário:** Altamente escalável.
- **Exemplo em BD:** Busca B-Tree!

Consulta $O(\log N)$:

```
-- 'cpf' possui um índice B-Tree
SELECT * FROM clientes
WHERE cpf = '123.456';
```

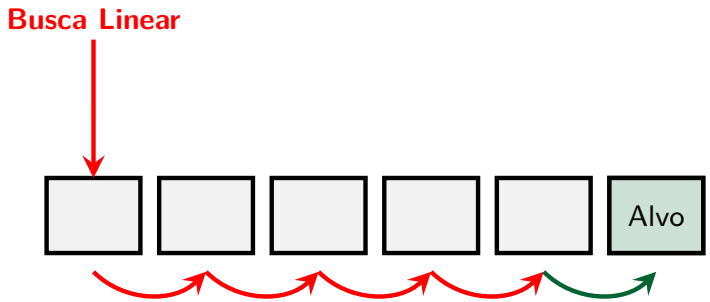


Revisão 5: $O(N)$ - Complexidade Linear

- O tempo cresce proporcionalmente.
- **Cenário:** Péssimo para Big Data.
- **Exemplo em BD:** Busca sem índice (*Full Table Scan*). Lê todas as linhas do disco.

Consulta $O(N)$:

```
-- Coluna sem índice!
SELECT * FROM clientes
WHERE nome LIKE '%Silva%';
```



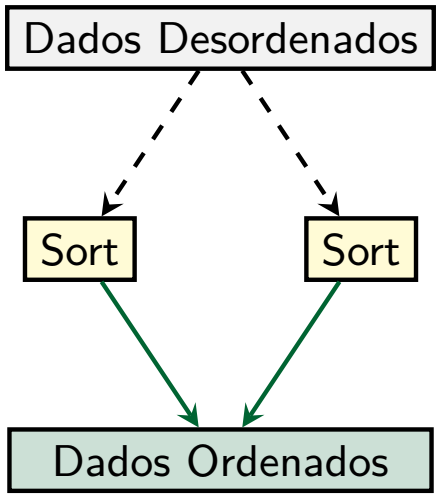
Lê bloco por bloco até o final!

Revisão 6: $O(N \log N)$ - Linearítmica

- Executa uma operação $\log N$ para cada um dos N itens.
- **Cenário:** Pior que Linear, mas é o melhor para **ordenar dados**.
- **Exemplo em BD:** Ordenar na memória colunas sem índice ou criar um índice novo.

Consulta $O(N \log N)$:

```
SELECT * FROM clientes
ORDER BY data_cadastro DESC;
```



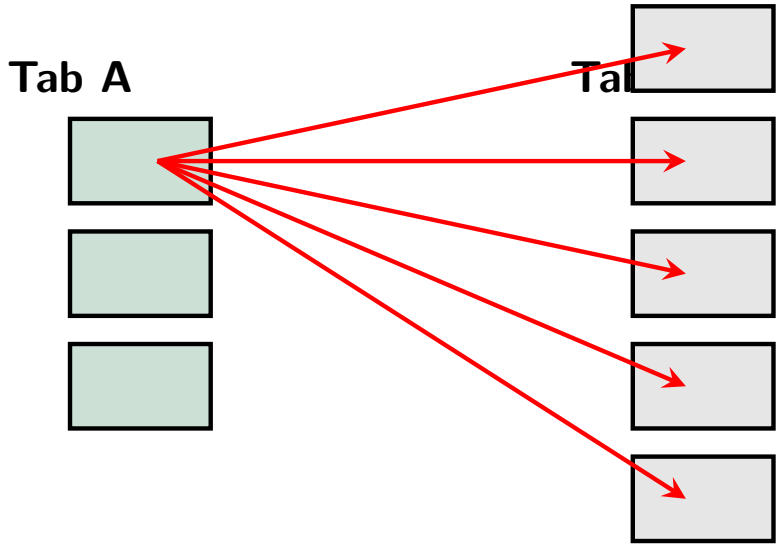
Custo em RAM!

Revisão 7: $O(N^2)$ - Complexidade Quadrática

- O tempo cresce ao quadrado. 1000 linhas = 1.000.000 de operações.
- **Cenário:** Inaceitável para BD.
- **Exemplo em BD:** *Nested Loop Join* sem índices. Para cada linha da Tab A, varre toda a Tab B!

Consulta $O(N^2)$:

```
SELECT * FROM t1
JOIN t2 ON t1.fk = t2.id;
-- Sem índice em fk e id!
```



Escaneia Tab B inteira para CADA linha da Tab A!

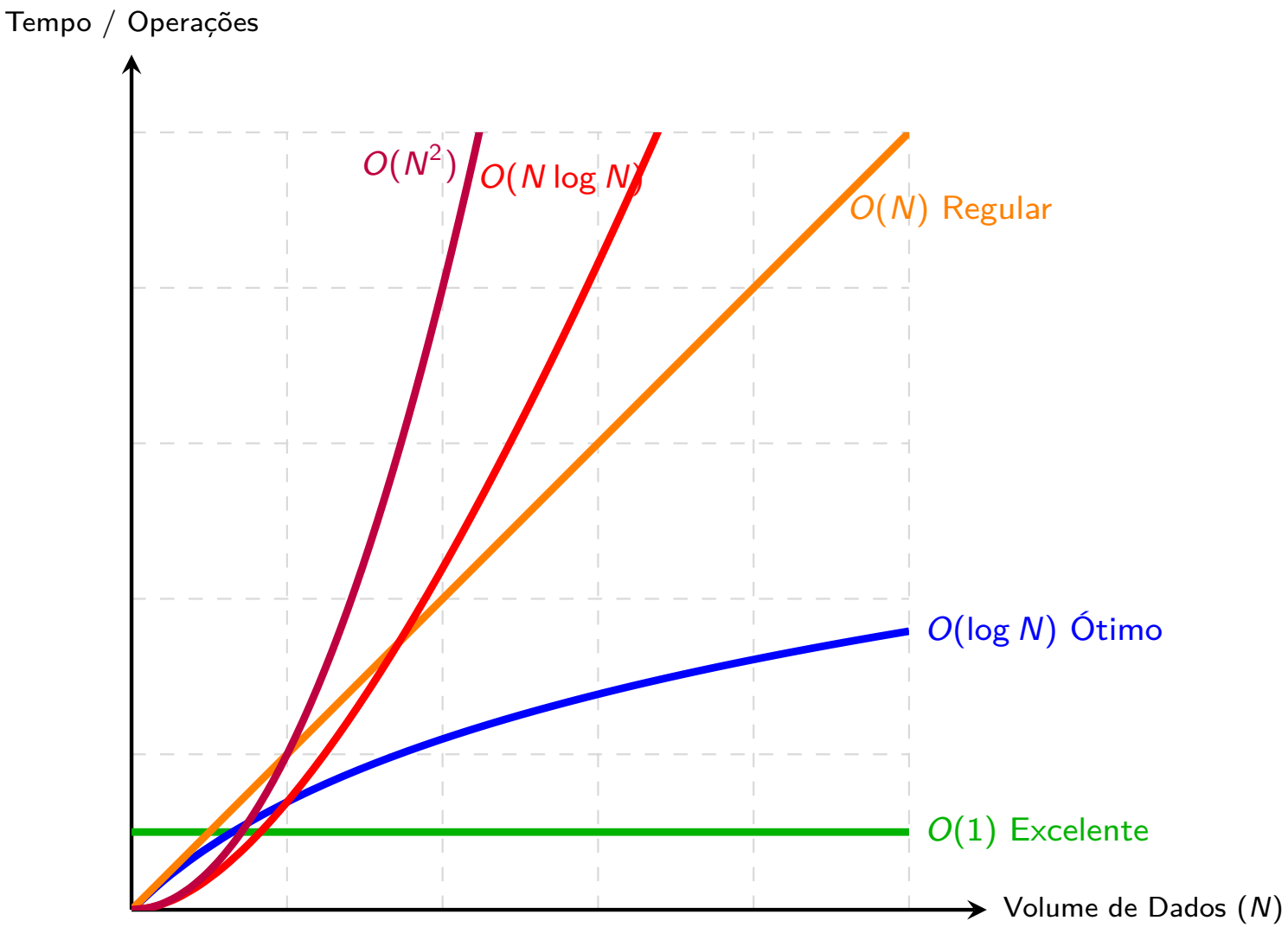
Revisão 8: Resumo Comparativo em BD

Imagine buscar um registro específico em uma tabela com 1 Milhão de linhas:

Complexidade	**Cenário (BD)**	**Operações Estimadas**
$O(1)$	Chave Hash (Ideal)	1
$O(\log N)$	**Busca com Índice (B-Tree)**	~ 20
$O(N)$	Busca sem Índice (Full Scan)	1.000.000
$O(N^2)$	JOIN sem índice (Nested Loop)	1.000.000.000.000

Conclusão: Índices $O(\log N)$ são a diferença técnica entre um sistema instantâneo e um sistema fora do ar!

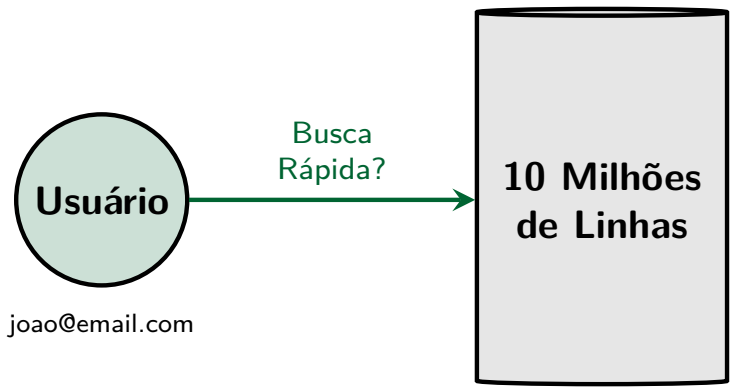
9. Comparação Visual das Complexidades



A Pergunta Inicial

"Como o banco encontra um registro específico em uma tabela com milhões de linhas?"

- Imagine um sistema com 10.000.000 de usuários.
- O usuário tenta fazer login, o sistema precisa verificar a senha daquele e-mail específico.
- Como a resposta é devolvida em frações de segundo?

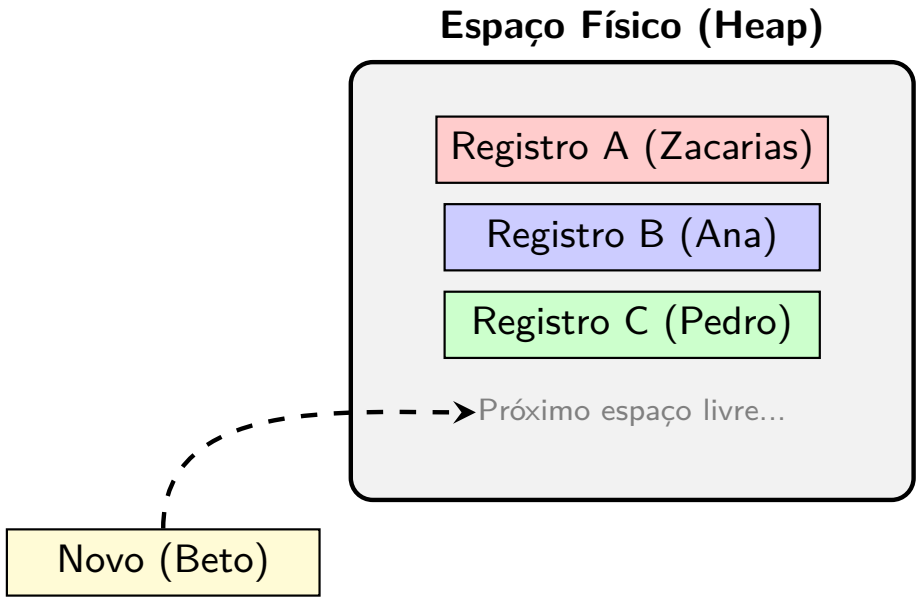


Conceitos Iniciais: O Cenário Padrão

Armazenamento Físico

Tabelas são conjuntos desordenados.

- Na prática física, quando você insere dados, eles são colocados no próximo espaço disponível no disco (estrutura chamada de *Heap*).
- Eles não têm uma ordem natural.

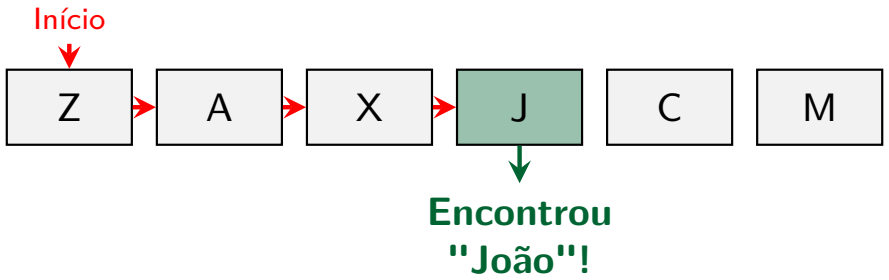


A Busca Sem Índice

Como o banco procura dados em um conjunto desordenado?

Busca Linear (Full Table Scan)

A **busca sem índice percorre todas as linhas** da tabela sequencialmente, do início ao fim, verificando se cada linha atende à condição da consulta.



- Tabela grande (milhões de linhas) $\rightarrow$ **Lentidão extrema**.

Discussão: Soluções do Cotidiano

Como resolvemos problemas semelhantes no mundo real?

Vamos refletir sobre duas situações cotidianas que lidam com grandes volumes de informação:

1. Encontrar um assunto específico em um livro grande.
2. Encontrar o número de uma pessoa na lista telefônica.

Analogia 1: O Índice de Livro

- **O Problema:** Você quer ler sobre "Normalização". Você lê página por página desde o sumário?

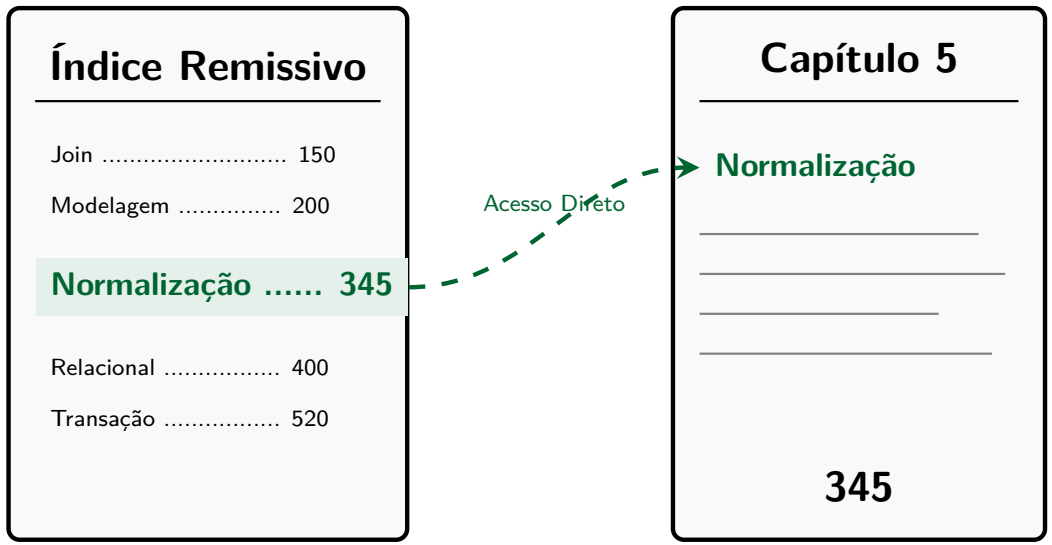
Analogia 1: O Índice de Livro

- **O Problema:** Você quer ler sobre "Normalização". Você lê página por página desde o sumário?

- **A Solução:** Você vai ao **Índice** no final do livro!
  - Ordenado alfabeticamente.
  - Encontra a palavra e a **página exata**.
  - Acesso direto à informação desejada.

Analogia 1: O Índice de Livro

- **O Problema:** Você quer ler sobre "Normalização". Você lê página por página desde o sumário?
- **A Solução:** Você vai ao **Índice** no final do livro!
  - Ordenado alfabeticamente.
  - Encontra a palavra e a **página exata**.
  - Acesso direto à informação desejada.



Analogia 2: A Lista Telefônica

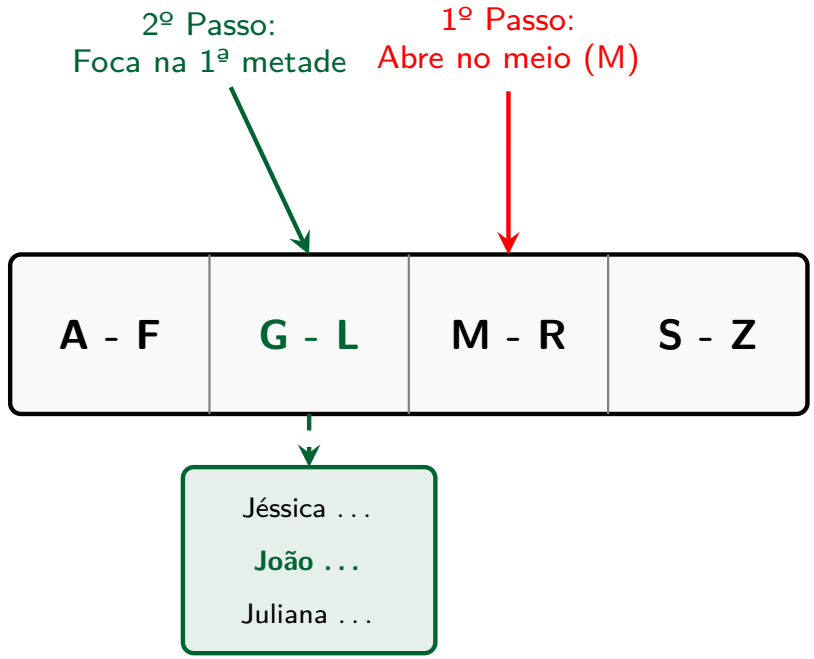
- **O Problema:** Achar o número do "João Silva".

Analogia 2: A Lista Telefônica

- **O Problema:** Achar o número do "João Silva".
- **A Solução:** A lista já está **ordenada alfabeticamente**.
  - Você não lê desde a letra A.
  - Abre no meio (letra M), sabe que J vem antes, foca na primeira metade e refina a busca.
  - Essa é a base lógica da busca em estruturas ordenadas (**busca binária** ou **árvores**).

Analogia 2: A Lista Telefônica

- **O Problema:** Achar o número do "João Silva".
- **A Solução:** A lista já está **ordenada alfabeticamente**.
  - Você não lê desde a letra A.
  - Abre no meio (letra M), sabe que J vem antes, foca na primeira metade e refina a busca.
  - Essa é a base lógica da busca em estruturas ordenadas (**busca binária** ou **árvores**).



Conclusão da Introdução

O Papel dos Índices

Da mesma forma que as analogias anteriores, **índices aceleram consultas** no banco de dados.

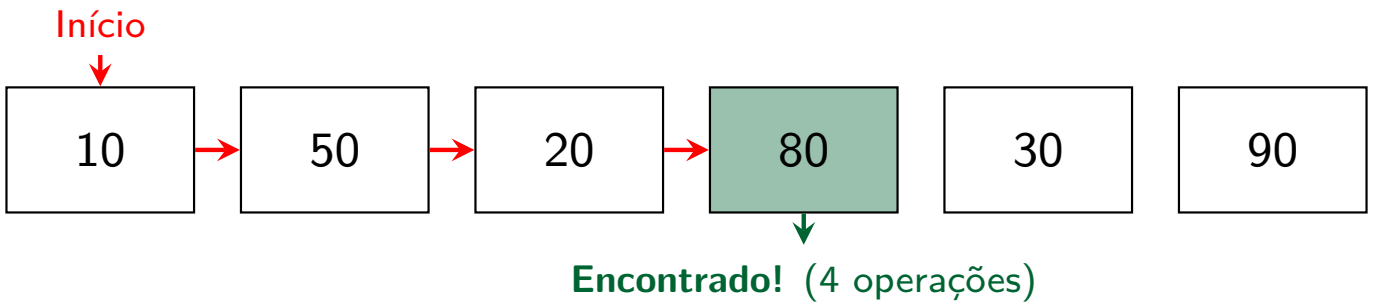
- O banco cria uma estrutura de dados adicional que mantém cópia de uma ou mais colunas de forma **ordenada**.
- Em vez de ler a tabela inteira (Busca Linear), o banco pesquisa no Índice rapidamente.
- O Índice aponta exatamente onde o registro real está no disco!

O Cenário sem Índice (Full Table Scan)

Imagine a busca:

```
SELECT * FROM Clientes WHERE id = 80;
```

Busca Linear: Em um *Heap* (dados desordenados), o banco lê linha por linha.

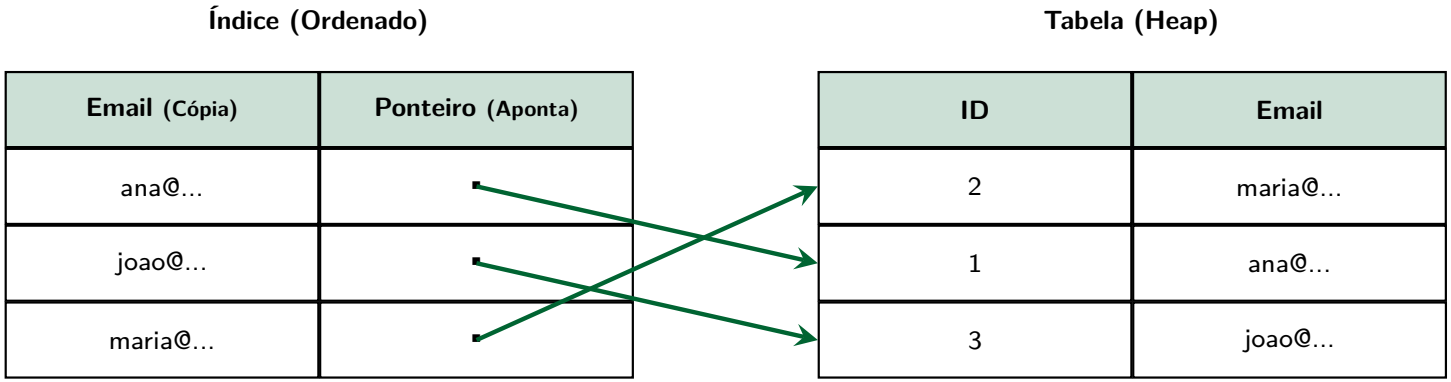


Criando o Índice na Prática

Comando SQL

```
CREATE INDEX idx_email
ON Clientes(email);
```

O índice é uma estrutura separada que armazena uma **cópia** dos dados (ordenada) e um **apontamento** para a linha real no disco.



A Estrutura Padrão do SQLite

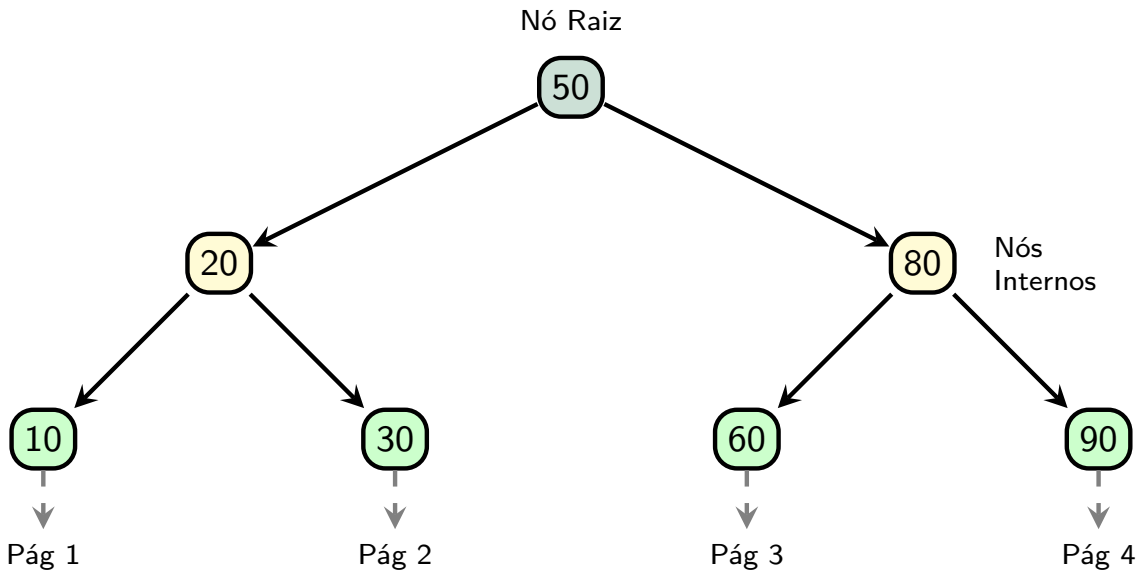
No **SQLite** (e em outros bancos relacionais), os índices usam a estrutura **B-Tree** (*Árvore B*).

- **Importante:** Não é uma Árvore Binária comum nem Rubro-Negra. A Árvore B é larga e rasa, projetada especificamente para minimizar acessos a blocos no disco.
- **Árvore Balanceada:** A distância da raiz até qualquer folha é sempre a mesma. Isso garante um tempo de busca previsível e muito rápido ($O(\log N)$).

Explicação Conceitual: Composição

Uma B-Tree possui:

- **Nós (Internos):** Funcionam como guias de busca. Contêm chaves que direcionam o caminho.
- **Folhas (Leaves):** Nível mais baixo. Contêm os apontamentos reais para os dados no disco.



Reorganização Automática

O que acontece ao fazer `INSERT` ou `DELETE`?

Manutenção do Balanceamento

A B-Tree se **reorganiza automaticamente** para garantir que continue perfeitamente balanceada e eficiente.

- **Split (Divisão):** Quando um nó fica cheio (atinge o limite de tamanho do bloco no SQLite), ele é dividido em dois e a árvore cresce para cima se necessário.
- **Merge (Fusão):** Quando muitos registros são apagados, os nós quase vazios são mesclados para economizar espaço de armazenamento e manter as rotas rápidas.

Operações Eficientes na B-Tree

A B-Tree é versátil. Ela é **otimizada para** resolver com extrema rapidez as seguintes operações no **SQLite**:

- **Igualdade (=):** Encontrar um único registro exato usando as chaves dos nós.
- **Intervalos (<, >, `BETWEEN`):** Ao achar o início, a estrutura permite caminhar lateralmente pelas folhas que já estão ordenadas.
- **Ordenação (`ORDER BY`):** Os dados na árvore já estão ordenados! O banco resolve consultas sem a necessidade de gastar memória extra.

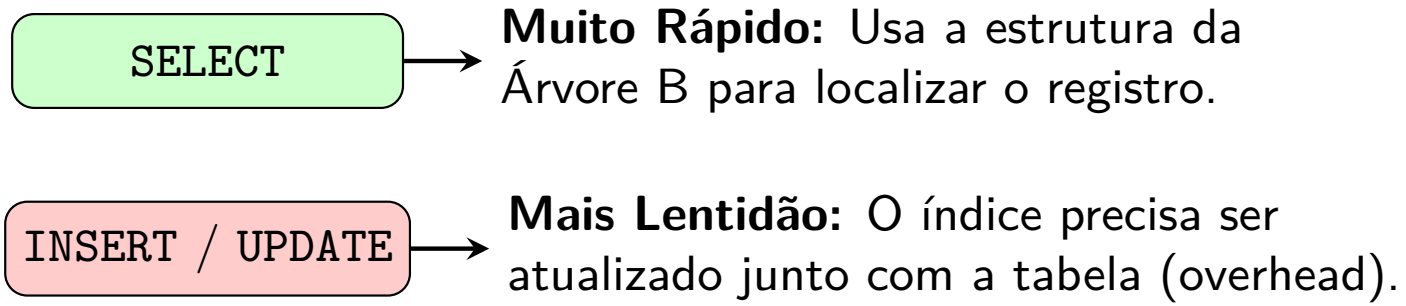
Operadores Otimizados pelo Índice B-Tree

=	<	>	BETWEEN	ORDER BY

Índices Compostos e Manutenção

```sql
CREATE INDEX idx_nome_cidade
ON Clientes (nome, cidade);
```

Manutenção de Índices: O Índice cobra seu preço!



Se o índice não for mais útil, podemos excluí-lo:

```sql
DROP INDEX idx_nome_cidade;
```

O comando EXPLAIN QUERY PLAN

- Como ter certeza de que o banco de dados está realmente usando o índice que criamos?
- O SQLite fornece o comando **EXPLAIN QUERY PLAN**.
- Ele não executa a consulta, apenas "lê a mente" do otimizador do banco e nos diz **como** ele planeja buscar os dados no disco.

Sintaxe

Basta adicionar as palavras antes de qualquer `SELECT`, `UPDATE` ou `DELETE`:

```
EXPLAIN QUERY PLAN
SELECT * FROM clientes WHERE email = 'joao@email.com';
```

Lendo o Resultado: SCAN vs SEARCH

O `EXPLAIN QUERY PLAN` devolve uma mensagem indicando o método de acesso escolhido pelo SQLite. Os dois mais comuns são:

SCAN TABLE

O que significa: Busca Linear $O(N)$.

- O banco vai ler **todas** as linhas da tabela.
- Sinal de alerta vermelho! Faltam índices ou a consulta está mal escrita.

SEARCH TABLE

O que significa: Busca Logarítmica $O(\log N)$.

- O banco vai usar um **Índice** (Árvore B).
- Caminho feliz! A consulta será incrivelmente rápida e eficiente.

Exemplo Prático: Tabela Sem Índice (SCAN)

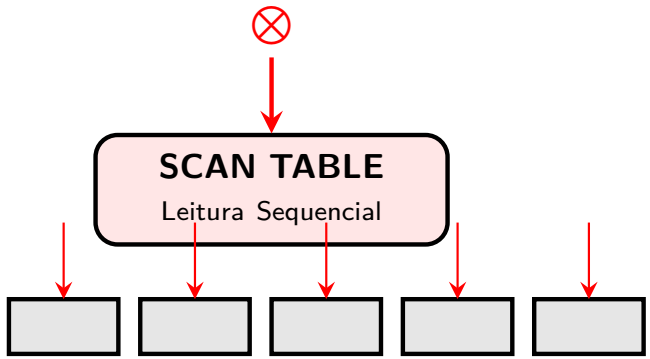
Consulta:

```
EXPLAIN QUERY PLAN
SELECT * FROM clientes
WHERE telefone = '555-1234';
```

Saída do SQLite:

```
SCAN TABLE clientes
```

Veredito: O banco fará um *Full Table Scan* $O(N)$. Desempenho péssimo para tabelas grandes.



Exemplo Prático: Com Índice (SEARCH)

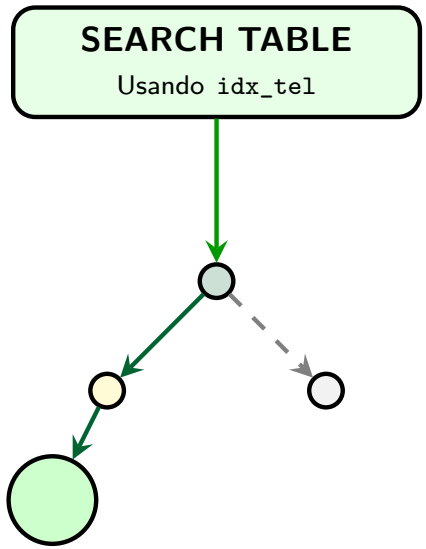
Criando o Índice e Consultando:

```
CREATE INDEX idx_tel
ON clientes(telefone);

EXPLAIN QUERY PLAN
SELECT * FROM clientes
WHERE telefone = '555-1234';
```

Saída do SQLite:

```
SEARCH TABLE clientes
USING INDEX idx_tel (telefone=?)
```



Veredito: O banco agora vai usar a Árvore B $O(\log N)$ do índice criado para pular direto para o dado!

PostgreSQL

PostgreSQL 1: A Extensibilidade de Índices

O **PostgreSQL** é mundialmente famoso pela sua extensibilidade e suporte nativo a múltiplos métodos de acesso a índices:

- **B-Tree:** O padrão confiável (para dados escalares comuns e comparações de ordenação).
- **Hash:** Totalmente focado em buscas exatas de igualdade ($O(1)$).
- **Expression & Partial:** Índices customizados para subconjuntos de dados ou cálculos específicos.
- **GIN (Generalized Inverted Index):** Para registros que contêm múltiplos valores internos.
- **GiST (Generalized Search Tree):** Para estruturas geométricas, espaciais, hierárquicas e faixas.
- **BRIN (Block Range Index):** Para tabelas gigantes ordenadas fisicamente (Big Data).

PostgreSQL 2: Índices HASH - Busca $O(1)$

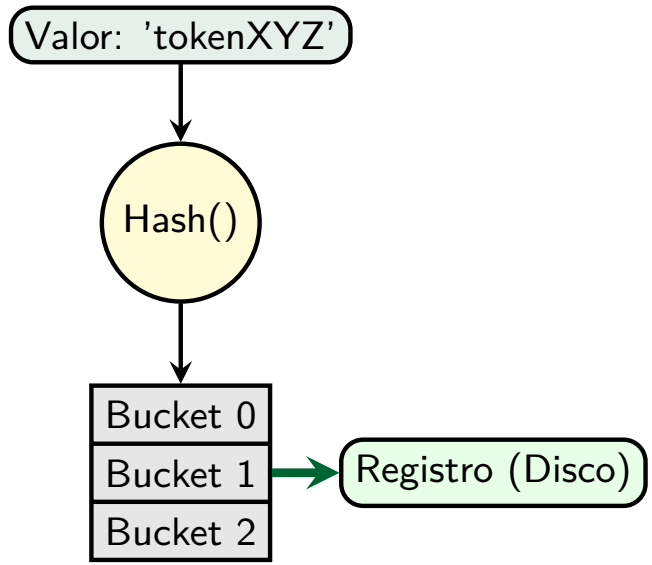
Diferente de bancos mais simples, o PostgreSQL possui suporte robusto a índices do tipo **Hash**:

- **Como Funciona:** Aplica uma função de espalhamento sobre a coluna. Mapeia diretamente o valor para o endereço do bloco físico ($O(1)$).
- **Limitação Importante:** Só funciona com o operador de igualdade (=). Não serve para operadores de comparação (<, >), intervalos ou ordenações.
- **Segurança (WAL):** Desde a versão 10, índices Hash são 100% seguros contra quedas de energia (WAL-logged) e replicáveis.

```
-- Criando um indice HASH
CREATE INDEX idx_hash_token
ON usuarios USING HASH (token_acesso);
```

PostgreSQL 3: HASH - Funcionamento Interno

- O valor do campo é transformado em um número inteiro (Hash).
- Esse hash aponta diretamente para um **Bucket** (balde).
- O balde contém o ponteiro direto da linha física (TID) no disco.
- Evita percorrer qualquer árvore, gerando velocidade instantânea!



PostgreSQL 4: Índices de Expressão

Por padrão, os índices olham apenas o valor bruto da coluna. O que acontece se filtrarmos usando funções?

- **O Problema:** Uma busca usando `LOWER(email)` ignora um índice normal criado na coluna `email`!
- **A Solução:** Indexar o resultado da própria função aplicada à coluna.

```
-- Criando indice de Expressao
CREATE INDEX idx_email_lower
ON clientes ( LOWER(email) );

-- Query super rapida O(log N) usando o indice:
SELECT * FROM clientes
WHERE LOWER(email) = 'ana@email.com';
```

PostgreSQL 5: Índices Parciais

Por que gastar disco indexando linhas que raramente ou nunca serão consultadas?

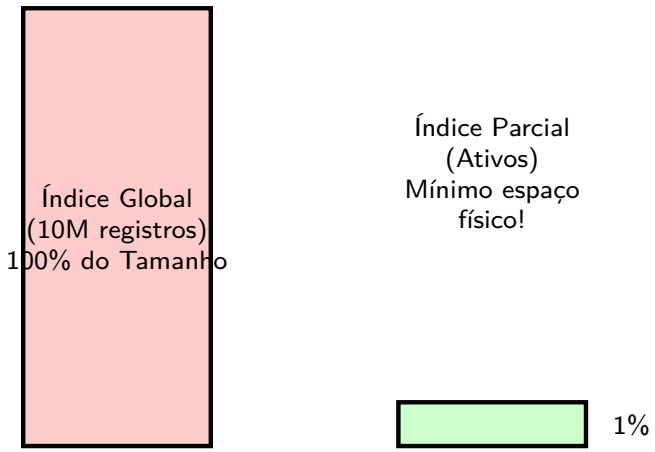
- **Conceito:** Um índice parcial contém apenas as referências para um subconjunto dos dados que atende a uma condição lógica (`WHERE`).
- **Vantagens:** Reduz drasticamente o tamanho do índice no disco e acelera as atualizações de escrita (`INSERT` / `UPDATE`).

```
-- Indexando apenas usuarios ativos
CREATE INDEX idx_usuarios_ativos
ON usuarios (email)
WHERE ativo = TRUE;

-- Otimiza perfeitamente queries com o mesmo filtro:
SELECT * FROM usuarios
WHERE email = 'admin@sistema.com' AND ativo = TRUE;
```

PostgreSQL 6: Índices Parciais - Economia de Recursos

- **Sem Índice Parcial:** O índice B-Tree guarda ponteiros para todos os 10 milhões de clientes, incluindo inativos e excluídos.
- **Com Índice Parcial:** Indexamos apenas os 1% de clientes ativos.
- Redução enorme em I/O e uso de RAM!



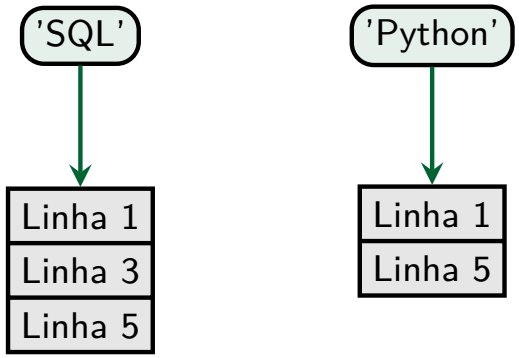
PostgreSQL 7: O que é o GIN?

O **GIN** (*Generalized Inverted Index*) é projetado para lidar com casos em que **um único registro contém múltiplos valores internos**.

- **Onde B-Tree Falha:** Uma árvore B assume uma relação de 1 valor chave por linha. Mas e se tivermos uma coluna com uma lista (Array) de tags ou um objeto JSON?
- **Abordagem GIN:** O GIN 'inverte' a relação. Ele mapeia os elementos individuais internos (ex: palavras, chaves, tags) de volta para todas as linhas (TIDs) que os contêm.
- **Usos principais:** Arrays indexados, documentos JSONB e buscas por texto completo.

PostgreSQL 8: GIN - Estrutura Interna

- Cada elemento interno (ex: 'SQL', 'Python') torna-se uma chave na Árvore do GIN.
- Abaixo de cada chave, existe uma lista (Postings List) contendo os IDs de todos os registros que contêm aquele valor.
- Encontrar dados em Arrays é incrivelmente rápido!



PostgreSQL 9: GIN - Arrays Indexados

Suponha que cada desenvolvedor cadastrado possua uma lista (Array) de tecnologias de interesse:

```
-- Tabela com coluna tipo Array
CREATE TABLE desenvolvedores (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100),
    tecnologias VARCHAR(50)[] -- Lista de stacks
);

-- Criando um indice GIN
CREATE INDEX idx_dev_techs
ON desenvolvedores USING GIN (tecnologias);

-- Query usando o operador de contencao (@>)
SELECT * FROM desenvolvedores
WHERE tecnologias @> ARRAY['PostgreSQL', 'Python'];
```

Nota: Sem o índice GIN, a busca com operador '@>' exigiria ler e varrer linearmente cada linha da tabela!

PostgreSQL 10: GIN - Documentos JSONB

O tipo `JSONB` armazena objetos JSON binários. Combinado com o GIN, entrega performance excelente para consultas semi-estruturadas:

```
CREATE TABLE produtos (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100),
    atributos JSONB
);

-- Indexando todas as chaves e valores internos do JSONB
CREATE INDEX idx_prod_atributos ON produtos USING GIN (atributos);

-- Busca direta no JSON com velocidade incrivel
SELECT * FROM produtos
WHERE atributos @> '{"categoria": "Eletro", "voltagem": "220v"}';
```

- **jsonb_ops (padrão):** Indexa todas as chaves, valores e subchaves do documento JSON.
- **jsonb_path_ops:** Ocupa menos espaço físico, porém focado em caminhos de navegação completos.

PostgreSQL 11: O que é o GiST?

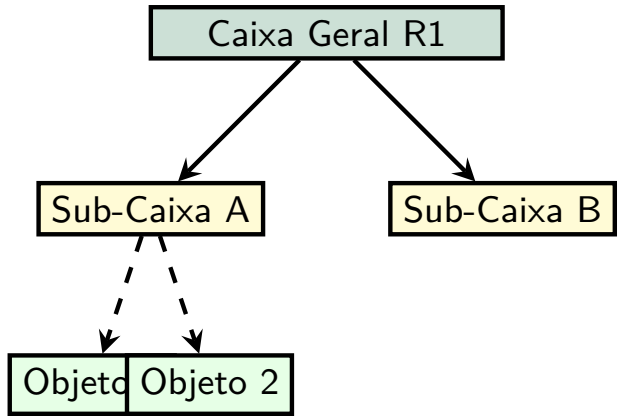
A árvore **GiST** (*Generalized Search Tree*) é um **template estrutural** altamente flexível.

- **Customização Total:** Permite definir novas regras de indexação para tipos de dados complexos que precisam ser buscados de forma balanceada.
- **Como Funciona:** Ao contrário de uma B-Tree tradicional que divide dados rigidamente de forma linear, a árvore GiST agrupa dados baseado em predicados de exclusão mútua (ex: 'contido em', 'sobreposto com').
- **Usos típicos:** Dados geográficos (coordenadas), intervalos de datas ou numéricos e buscas textuais especiais.

PostgreSQL 12: GiST - A Estrutura Hierárquica

A árvore do GiST é estruturada em torno de caixas delimitadoras ou condições que podem se sobrepor:

- **Nós Superiores:** Guardam predicados gerais abrangentes (ex: 'Região Centro-Oeste').
- **Nós Inferiores:** Refinam a busca para sub-regiões (ex: 'Goiás' -> 'Cidade de Rio Verde').
- Isso permite descartar ramos inteiros de busca no disco instantaneamente!



PostgreSQL 13: Índices Espaciais (PostGIS)

O ecossistema de processamento espacial do PostgreSQL (**PostGIS**) depende fundamentalmente dos índices do tipo GiST:

- **Desafio:** Coordenadas no mapa (Latitude, Longitude) não possuem ordem linear. Como indexar duas dimensões geográficas?
- **Solução:** O GiST agrupa dados geométricos usando **Bounding Boxes** (Retângulos Mínimos Envolventes).

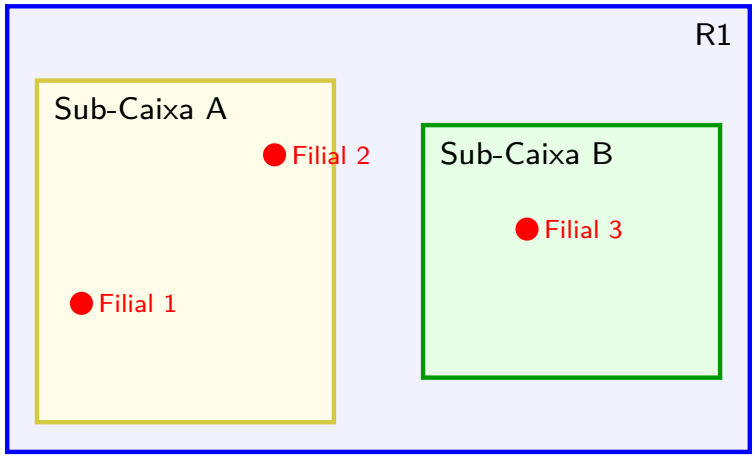
```
-- Habilitando extensao espacial no PostgreSQL
CREATE EXTENSION postgis;

-- Tabela com dados geometricos
CREATE TABLE filiais (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100),
    localizacao GEOMETRY(Point, 4326) -- Tipo Ponto Geografico
);

-- Indispensavel para consultas geometricas espaciais rapidas!
CREATE INDEX idx_filiais_geo ON filiais USING GIST (localizacao);
```

PostgreSQL 14: GiST / R-Tree em Ação

Visualizando o agrupamento geográfico em duas dimensões (R-Tree):



O banco localiza a Filial 3 navegando apenas pela Sub-Caixa B, ignorando totalmente a Sub-Caixa A!

PostgreSQL 15: O que são Trigramas?

O módulo **pg_trgm** introduz uma técnica poderosa baseada em **Trigramas** para resolver buscas parciais textuais flexíveis.

- **A Limitação do B-Tree:** Um índice comum B-Tree só serve para buscas com curinga no final (`LIKE 'Silva%'`). Se usarmos `LIKE '%Silva%'`, o índice B-Tree é ignorado.
- **O Conceito do Trigrama:** Qualquer texto é quebrado em pequenas fatias sobrepostas de 3 letras consecutivas.
- **Exemplo:** A palavra 'BANCO' gera os trigramas:

  `' b'`, `' ba'`, `'ban'`, `'anc'`, `'nco'`, `'co '`

- **Poder Máximo:** Permite buscas ultra rápidas mesmo com termos no início e fim (`%termo%`) e até correção ortográfica (Fuzzy Match)!

PostgreSQL 16: Trigramas em Ação com GIN

Instalando a extensão e criando um índice que armazena os trigramas das strings:

```
-- Ativando o modulo oficial de trigramas
CREATE EXTENSION pg_trgm;

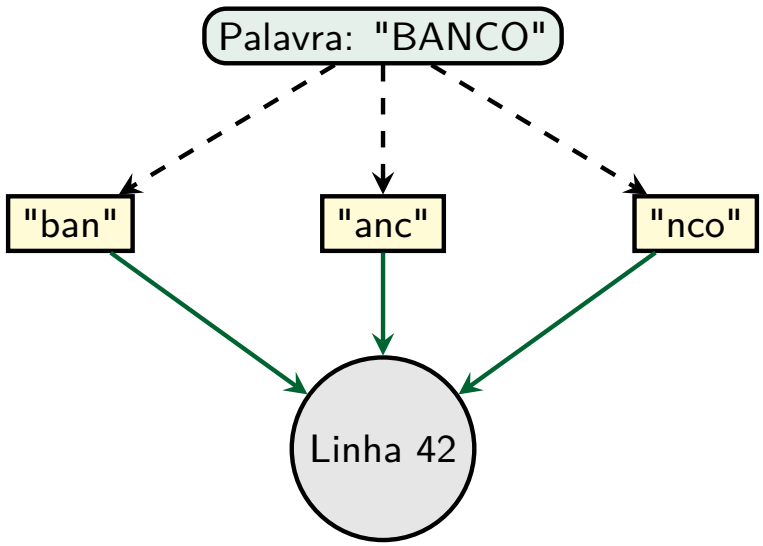
CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100)
);

-- Criando indice GIN usando a classe de operador de trigramas
CREATE INDEX idx_usuarios_nome_trgm
ON usuarios USING GIN (nome gin_trgm_ops);

-- Consulta O(log N) altamente otimizada, mesmo com % no inicio!
SELECT * FROM usuarios
WHERE nome LIKE '%Sergio%';
```

PostgreSQL 17: Como os Trigramas São Mapeados

Funcionamento interno do mapeamento entre trigramas e registros no disco:



Ao buscar por '%banco%', o banco decompõe a palavra em trigramas, intersecta suas listas no índice GIN e encontra a **Linha 42** de forma instantânea!

PostgreSQL 18: Busca Textual Avançada (Full-Text Search)

A busca por texto completo nativa do PostgreSQL é capaz de substituir sistemas externos inteiros (como Elasticsearch) em projetos de médio porte:

- **tsvector:** Tipo de dados que representa um documento analisado, aplicando radicalização (*stemming*) e eliminando palavras irrelevantes (*stop words* como 'e', 'de', 'um').
- **tsquery:** Representa a nossa consulta contendo operadores booleanos (`&` (E), `|` (OU), `!` (NÃO)).
- **Exemplo:** As palavras 'correndo', 'corri', 'correu' são todas simplificadas e convertidas para o mesmo radical: `'corr'`.

PostgreSQL 19: FTS na Prática

Implementando busca de artigos usando indexação por texto completo com o GIN:

```
CREATE TABLE artigos (
    id SERIAL PRIMARY KEY,
    titulo VARCHAR(200),
    conteudo TEXT
);

-- Criando indice GIN usando a funcao to_tsvector em portugues
CREATE INDEX idx_artigos_fts
ON artigos USING GIN (to_tsvector('portuguese', conteudo));

-- Query extremamente rapida usando o operador @@
SELECT * FROM artigos
WHERE to_tsvector('portuguese', conteudo) @@ to_tsquery('portuguese', 'bancos & indices');
```

- A query acima busca e encontra artigos que contêm radicais de 'bancos' e 'índices', mesmo em flexões gramaticais diferentes.

PostgreSQL 20: Matriz de Decisão de Índices

Método	Complexidade	Melhor Operador	Cenário Ideal (BD)
B-Tree	$O(\log N)$	=, <, >, `BETWEEN`	Dados comuns ordenáveis (Datas, IDs, Nomes).
Hash	$O(1)$	=	Chaves de token, UUIDs (busca exata).
GIN	$O(\log N)$	`@>`, `?`, `@@`	Documentos JSONB, Arrays, Busca Textual (FTS).
GiST	$O(\log N)$	`&&`, «, »	Dados Geográficos (PostGIS), Intervalos numéricos/datas.
BRIN	$O(N)$ parcial	Limites de bloco	Logs gigantes ou dados históricos ordenados fisicamente.

Dica de Ouro: Use sempre o comando `EXPLAIN ANALYZE` no PostgreSQL para certificar suas escolhas de índice em ambiente de produção!